STACK

IN DSA

LIFO DATA STRUCTURE

LIFO
LAST IN, FIRST OUT

PUSH
ADD ELEMNT

POP
REMOVE ELEMENT

What is Stack?

- A Stack is a linear data structure that follows the rule:
  - **LIFO (Last In First Out)**
- **Meaning:**
  - The element inserted last is removed first.
  - Insertion and deletion happen from only one end called **TOP**.
- **Real Life Examples:**
  - Stack of plates
  - Browser history
  - Undo operation in MS Word

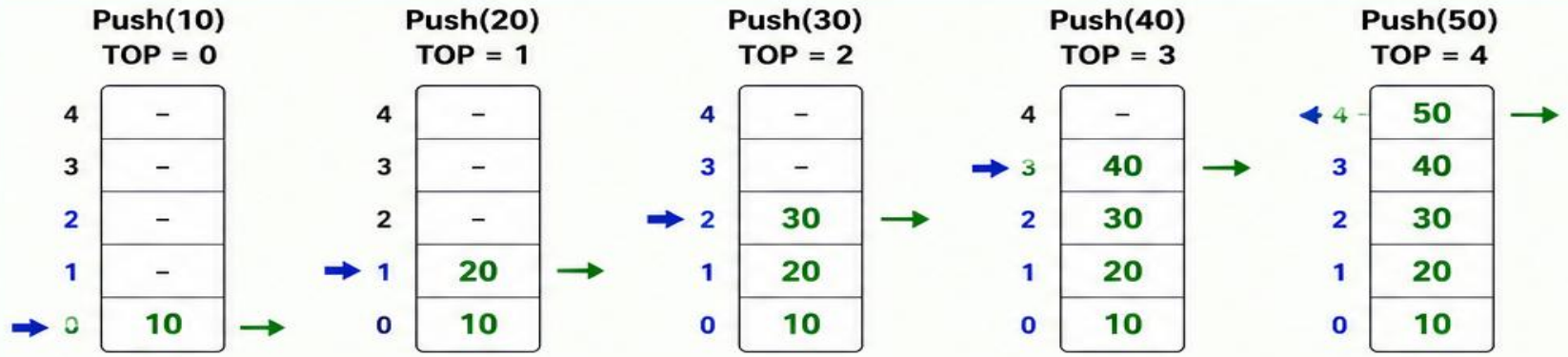
STACK OPERATIONS: PUSH, POP AND OVERFLOW (EXAMPLE)

STACK CAPACITY (SIZE) = 5 | **INITIAL STATE: EMPTY STACK** | **TOP = −1 (When Stack is Empty)**

PUSH OPERATIONS

Push: Add element at the top of stack.

TOP → Newly Inserted Element →



OVERFLOW SCENARIO

Stack is Full (TOP = 4)

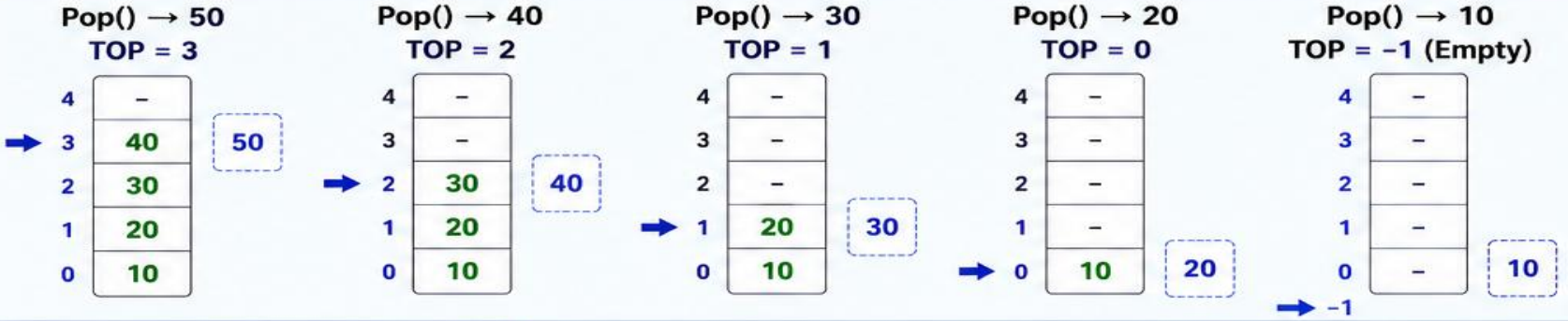
If we try to Push(60)

↓

OVERFLOW
(Insert Operation cannot be performed)

POP OPERATIONS

Pop: Remove element from the top of stack.



UNDERFLOW SCENARIO

Stack is Empty (TOP = −1)

If we try to Pop()

↓

UNDERFLOW
(Delete Operation cannot be performed)

SUMMARY TABLE

Operation	Element	TOP (After Operation)	Stack Status (Bottom → Top)
Push	10	0	10
Push	20	1	10, 20
Push	30	2	10, 20, 30
Push	40	3	10, 20, 30, 40
Push	50	4	10, 20, 30, 40, 50
Pop	50	3	10, 20, 30, 40
Pop	40	2	10, 20, 30
Pop	30	1	10, 20
Pop	20	0	10
Pop	10	−1	Empty Stack

KEY POINTS

- Stack follows **LIFO (Last In First Out)** principle.
- **Push:** Insert element at the top and update TOP.
- **Pop:** Remove element from the top and update TOP.
- **Overflow:** When TOP reaches SIZE − 1 and we try to PUSH.
- **Underflow:** When TOP is −1 and we try to POP.

STACK VISUAL INDEX

Program

```python
# Stack Implementation using Functions
stack = []
# Push Function
def push():
    item = input("Enter element to push: ")
    stack.append(item)
    print(item, "pushed into stack.")
# Pop Function
def pop():
    if len(stack) == 0:
        print("Stack Underflow! Stack is empty.")
    else:
        print("Popped element:", stack.pop())
# Display Function
def display():
    if len(stack) == 0:
        print("Stack is empty.")
    else:
        print("Stack elements (Top to Bottom):")
        for i in range(len(stack) - 1, -1, -1):
            print(stack[i])
```

```python
# Main Program
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        push()

    elif choice == 2:
        pop()

    elif choice == 3:
        display()

    elif choice == 4:
        print("Program Ended.")
        break

    else:
        print("Invalid Choice!")

```

Applications of Stack

- Function Calling
- Recursion
- Expression Evaluation
- Undo/Redo
- Browser Back Button
- Syntax Parsing
- Parentheses Matching
- Expression Conversion

Type of Expression

- Infix, Prefix and Postfix

Type	Example
Infix	A + B
Prefix	+AB
Postfix	AB+

Rules for Converting Infix to Postfix

- **1. If Operand Comes**
  - Directly add it to postfix expression.
- **2. If Left Parenthesis (Comes**
  - Push it into stack.
- **3. If Right Parenthesis) Comes**
  - Pop and print elements from stack until (is found.
  - Remove (from stack.
- **4. If Operator Comes (+, -, *, /, ^)**
  - Compare precedence of current operator with top of stack.
  - If stack operator has higher or equal precedence:
    - Pop operator from stack and add to postfix.
  - Then push current operator into stack.

Algorithm: Convert Infix Expression to Postfix

1. Start.
2. Create an empty stack.
3. Scan the infix expression from left to right.
4. If the scanned symbol is an operand, add it to the postfix expression.
5. If the scanned symbol is (, push it into the stack.
6. If the scanned symbol is), pop operators from the stack and add them to postfix until (is found. If the scanned symbol is an operator:
   a) Pop operators from the stack while they have higher or equal precedence.
   b) Add popped operators to postfix.
   c) Push the current operator into the stack.
7. Repeat steps 4 to 7 until the expression ends.
8. Pop all remaining operators from the stack and add them to postfix.
9. Stop.

Program

```
# Function to return precedence of operators
def precedence(op):
    if op == '+' or op == '-':
        return 1
    elif op == '*' or op == '/':
        return 2
    elif op == '^':
        return 3
    return 0

def infix_to_postfix(exp):
    stack = []
    postfix = ""

    for ch in exp:
        # Operand
        if ch.isalnum():
            postfix += ch
        # Left Parenthesis
        elif ch == '(':
            stack.append(ch)
        # Right Parenthesis
        elif ch == ')':
            while stack and stack[-1] != '(':
                postfix += stack.pop()
            stack.pop()  # Remove '('

        # Operator
        else:
            while (stack and stack[-1] != '(' and
                   precedence(stack[-1]) >= precedence(ch)):
                postfix += stack.pop()
            stack.append(ch)
    # Pop remaining operators
    while stack:
        postfix += stack.pop()

    return postfix

# Main Program
exp = input("Enter Infix Expression: ")
print("Postfix Expression:", infix_to_postfix(exp))
```

INFIX TO POSTFIX CONVERSION USING STACK (STEP-WISE)

Infix Expression: A + B * (C - D) / E

Postfix Expression: A B C D - * E / +

Step	Scanned Symbol	Action	Stack (Top → Right)	Postfix Expression
1	A	Operand → Add to output	Ø	A
2	+	Operator → Push to stack	+	A
3	B	Operand → Add to output	+	A B
4	*	Operator → Higher precedence → Push to stack	+ *	A B
5	(	Left Parenthesis → Push to stack	+ * (	A B
6	C	Operand → Add to output	+ * (	A B C
7	-	Operator → Push to stack	+ * (-	A B C
8	D	Operand → Add to output	+ * (-	A B C D
9	)	Right Parenthesis → Pop until '('	+ * **Pop -** → + * **Pop (** → + *	A B C D -
10	/	Operator → Lower precedence than '*' Pop until lower precedence	+ * **Pop *** → + **Push (** → + /	A B C D - *
11	E	Operand → Add to output	+ /	A B C D - * E
12	(End)	End of expression → Pop all from stack	+ / **Pop /** → + **Pop +** → Ø	A B C D - * E / +

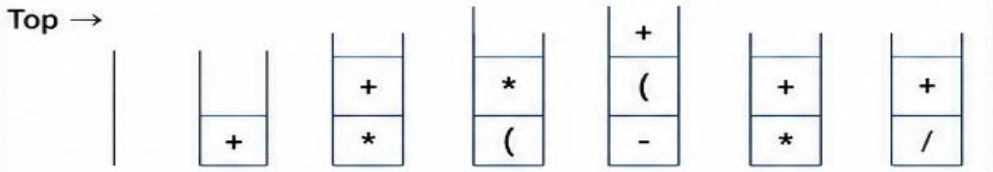
Precedence:

(→ Lowest
+ - → 1
* / → 2
) → Highest

Rules:

- If the symbol is an operand, add it to output.
- If the symbol is an operator, push it to stack according to precedence.
- If the symbol is '(', push it to stack.
- If the symbol is ')', pop from stack and add to output until '(' is found.
- At the end, pop all remaining operators from stack and add to output.

Stack Visualization Example



INFIX TO POSTFIX CONVERSION USING STACK (STEP-WISE)

Infix Expression: (A + B) * C - D / (E - F)

Postfix Expression: A B + C * D E F - / -

Step	Scanned Symbol	Action	Stack (Top → Right)	Postfix Expression
1	(	Left Parenthesis → Push to stack	(	–
2	A	Operand → Add to output	(	A
3	+	Operator → Push to stack	(+	A
4	B	Operand → Add to output	(+	A B
5	)	Right Parenthesis → Pop until '('	Pop + → (	A B +
6	*	Operator → Push to stack	*	A B +
7	C	Operand → Add to output	*	A B + C
8	-	Operator → Lower precedence than '*' Pop until lower precedence	Pop * → -	A B + C *
9	D	Operand → Add to output	-	A B + C * D
10	/	Operator → Push to stack	- /	A B + C * D
11	(	Left Parenthesis → Push to stack	- / (	A B + C * D
12	E	Operand → Add to output	- / (	A B + C * D E
13	-	Operator → Push to stack	- / (-	A B + C * D E
14	F	Operand → Add to output	- / (-	A B + C * D E F
15	)	Right Parenthesis → Pop until '('	Pop - → /	A B + C * D E F -
16	(End)	End of expression → Pop all from stack	Pop / → -	A B + C * D E F - / -
17			Pop - → Ø	

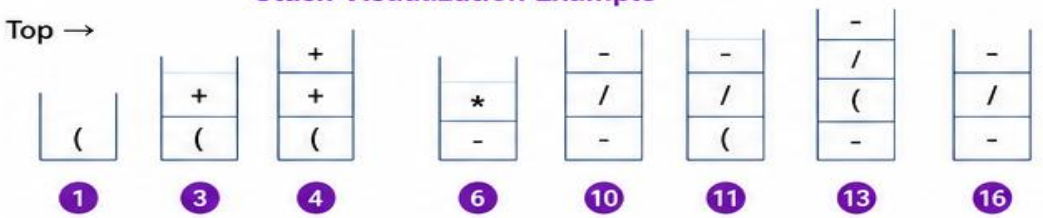
Precedence:

(→ Lowest
+ - → 1
* / → 2
) → Highest

Rules:

- If the symbol is an operand, add it to output.
- If the symbol is an operator, push it to stack according to precedence.
- If the symbol is '(', push it to stack.
- If the symbol is ')', pop from stack and add to output until '(' is found.
- At the end, pop all remaining operators from stack and add to output.

Stack Visualization Example



INFIX TO POSTFIX CONVERSION USING STACK (STEP-WISE)

Infix Expression: A * (B + C) - D / E

Postfix Expression: A B C + * D E / -

Step	Scanned Symbol	Action	Stack (Top → Right)	Postfix Expression
1	A	Operand → Add to output	Ø	A
2	*	Operator → Push to stack	*	A
3	(	Left Parenthesis → Push to stack	* (	A
4	B	Operand → Add to output	* (	A B
5	+	Operator → Push to stack	* (+	A B
6	C	Operand → Add to output	* (+	A B C
7	)	Right Parenthesis → Pop until '('	Pop + → * (Pop (→ *	A B C +
8	-	Operator → Lower precedence than '*' Pop until lower precedence	Pop * → Ø Push - → -	A B C + *
9	D	Operand → Add to output	-	A B C + * D
10	/	Operator → Push to stack	- /	A B C + * D
11	E	Operand → Add to output	- /	A B C + * D E
12	(End)	End of expression → Pop all from stack	Pop / → -	A B C + * D E / -
13			Pop - → Ø	

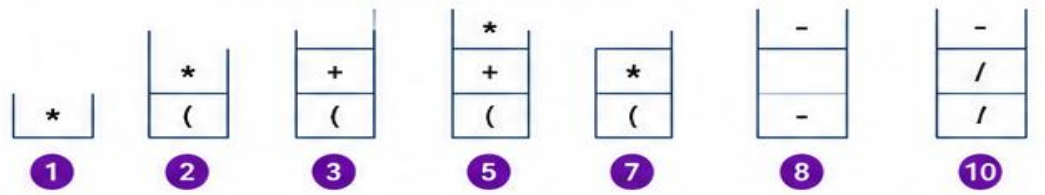
Precedence:

(→ Lowest
+ - → 1
* / → 2
) → Highest

Rules:

- If the symbol is an operand, add it to output.
- If the symbol is an operator, push it to stack according to precedence.
- If the symbol is '(', push it to stack.
- If the symbol is ')', pop from stack and add to output until '(' is found.
- At the end, pop all remaining operators from stack and add to output.

Stack Visualization Example (Top →)



Rules for Evaluating Postfix Expression

- **1. Scan Expression from Left to Right**
  - Read one symbol at a time.
- **2. If Operand Comes**
  - Push operand into stack.
- **If Operator Comes**
  - Pop two operands from stack.
  - Perform operation.
  - Push result back into stack.
- **Important**
- First popped element = Operand 2
- Second popped element = Operand 1

Algorithm for Postfix Evaluation

- Create an empty stack.
- Scan postfix expression from left to right.
- If operand:
  - Push into stack.
- If operator:
  - Pop two operands.
  - Apply operation.
  - Push result into stack.
- Repeat until expression ends.
- Final value in stack is the answer.

EVALUATION OF POSTFIX EXPRESSION USING STACK (STEP BY STEP)

Postfix Expression: 8 2 3 + 5 * 7 -

Explanation: Scan from left to right.
If operand → Push into stack
If operator → Pop two operands, apply operator (first popped = b, second = a), compute (a operator b) and push result back into stack.

Step	Scanned Symbol	Type	Action / Explanation	Stack Status (Bottom → Top)	Calculation
1	8	Operand	Push 8	8	–
2	2	Operand	Push 2	8 2	–
3	3	Operand	Push 3	8 2 3	–
4	+	Operator	Pop 3 and 2 → (2 + 3) = 5 Push 5	8 5	a = 2, b = 3 2 + 3 = 5
5	5	Operand	Push 5	8 5 5	–
6	*	Operator	Pop 5 and 5 → (5 * 5) = 25 Push 25	8 25	a = 5, b = 5 5 * 5 = 25
7	7	Operand	Push 7	8 25 7	–
8	-	Operator	Pop 7 and 25 → (25 - 7) = 18 Push 18	8 18	a = 25, b = 7 25 - 7 = 18
9	(End)	End	End of expression Top of stack is the final result.	18	Final Result = 18

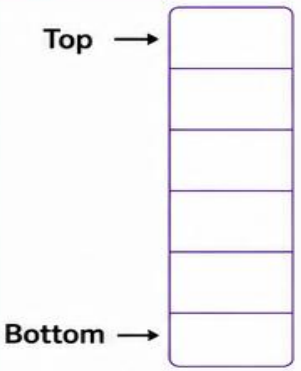
Final Result (Top of Stack) = 18

So, the value of the postfix expression is 18.

HOW IT WORKS

1. If symbol is an operand, push it into stack.
2. If symbol is an operator, pop two operands from stack.
3. Apply operator on them (first popped = b, second popped = a).
4. Push the result back into stack.
5. At the end, top of the stack is the final result.

STACK VISUAL GUIDE



QUICK REFERENCE

Operator	Operation	Example (a op b)
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b

SAME EXPRESSION – STEP HIGHLIGHTS

Postfix Expression: 8 2 3 + 5 * 7 -

Main Steps:

→ 2 + 3 = 5

→ 5 * 5 = 25

→ 25 - 7 = 18 (Final Result)

NOTES

Postfix evaluation always follows LIFO (Last In First Out).

- Always pop in order: first b (top), then a (next).
- Works for all +, -, *, / operations.
- Time Complexity: O(n)
- Space Complexity: O(n)

EVALUATION OF POSTFIX EXPRESSION USING STACK (STEP BY STEP)

Postfix Expression: 6 3 2 + 5 * 8 - 4 /

Explanation: Scan from left to right.
If operand → Push into stack
If operator → Pop two operands, apply operator (first popped = b, second = a), compute (a operator b) and push result back into stack.

Step	Scanned Symbol	Type	Action / Explanation	Stack Status (Bottom → Top)	Calculation
1	6	Operand	Push 6	6	–
2	3	Operand	Push 3	6 3	–
3	2	Operand	Push 2	6 3 2	–
4	+	Operator	Pop 2 and 3 → (3 + 2) = 5 Push 5	6 5	a = 3, b = 2 3 + 2 = 5
5	5	Operand	Push 5	6 5 5	–
6	*	Operator	Pop 5 and 5 → (5 * 5) = 25 Push 25	6 25	a = 5, b = 5 5 * 5 = 25
7	8	Operand	Push 8	6 25 8	–
8	-	Operator	Pop 8 and 25 → (25 - 8) = 17 Push 17	6 17	a = 25, b = 8 25 - 8 = 17
9	/	Operator	Pop 17 and 6 → (6 / 17) = 0 (Integer Div.) Push 0	0	a = 6, b = 17 6 / 17 = 0
10	(End)	End	End of expression Top of stack is the final result.	0	Final Result = 0

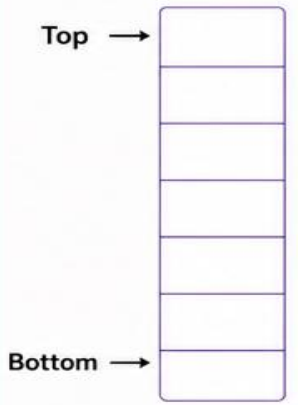
Final Result (Top of Stack) = 0

So, the value of the postfix expression is 0.

HOW IT WORKS

1. If symbol is an operand, push it into stack.
2. If symbol is an operator, pop two operands from stack.
3. Apply operator on them (first popped = b, second popped = a).
4. Push the result back into stack.
5. At the end, top of the stack is the final result.

STACK VISUAL GUIDE



QUICK REFERENCE

Operator	Operation	Example (a op b)
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b

SAME EXPRESSION – STEP HIGHLIGHTS

Postfix Expression: 6 3 2 + 5 * 8 - 4 /
Main Steps:
→ 3 + 2 = 5
→ 5 * 5 = 25
→ 25 - 8 = 17
→ 6 / 17 = 0 (Integer Division)

NOTES

- Postfix evaluation follows LIFO (Last In First Out).
- Always pop in order: first b (top), then a (next).
- Works for all +, -, *, / operations.
- Time Complexity: O(n)
- Space Complexity: O(n)

Program

```
# Function to evaluate postfix expression
def evaluate_postfix(exp):
    stack = []
    for ch in exp:
        # If operand, push to stack
        if ch.isdigit():
            stack.append(int(ch))
        else:
            b = stack.pop()
            a = stack.pop()

            if ch == '+':
                stack.append(a + b)
            elif ch == '-':
                stack.append(a - b)
            elif ch == '*':
                stack.append(a * b)
            elif ch == '/':
                stack.append(a / b)
            elif ch == '^':
                stack.append(a ** b)
    return stack.pop()
```

```
# Main Program
postfix = input("Enter Postfix Expression: ")
result = evaluate_postfix(postfix)
print("Result =", result)
```